

Stage 1.4.2.3 — Deep PDF Page Analysis

Absolutely. We are at **Stage 1.4.2.3 — Deep PDF Page Analysis**.

This is a very important step because we're moving from: *'Does this page contain text?'* to: *'What exactly is inside this PDF page, and does the extracted content look trustworthy?'*

We will continue using your single notebook:

DIRECTORY

```
notebooks/
├── text-loaders/
│   └── 01_data_ingestion.ipynb
```

And importantly, we still will not install OCR.

1. What are we going to inspect?

For every PDF page, we'll collect:

INSPECTION HIERARCHY

```
PDF Page
├── Text
│   ├── character count
│   ├── word count
│   └── text blocks
├── Images
│   ├── image count
│   └── image information
├── Layout
│   ├── page dimensions
│   └── block information
└── Extraction signals
    ├── text density
    └── possible scanned page
```

This will give us a much better foundation for deciding whether a page should go through:

DECISION PATH

```
Normal extraction
OR
OCR
```

2. Why are we going below LangChain?

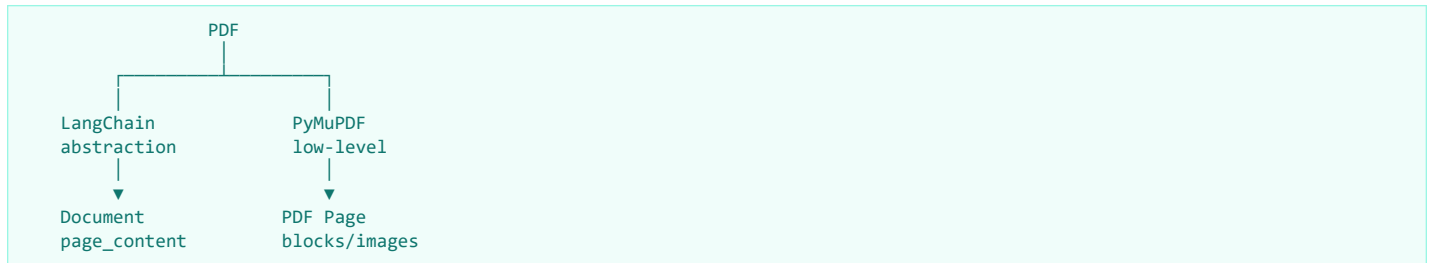
Until now we've been doing:

ABSTRACTION CHAIN

```
PyMuPDFLoader
┆
┆ LangChain Document
┆
┆ page_content
```

That's excellent for normal RAG development. But for document inspection, we need to look at the underlying PDF itself. Conceptually:

DUAL-LAYER MODEL



We are temporarily using PyMuPDF directly because it gives us information that the LangChain Document abstraction intentionally hides. This isn't replacing LangChain. It's helping us understand what happens underneath the abstraction.

3. Import PyMuPDF

Add a new Markdown cell:

MARKDOWN

1.4.2.3 Deep PDF Page Analysis

In this experiment we inspect the underlying PDF pages using PyMuPDF to understand text blocks, images, page dimensions, text density, and other signals that can help identify potentially scanned or complex pages.

Then:

PYTHON

```
import fitz
```

PyMuPDF is imported as fitz.

4. Open the text-based PDF

We already have `text_pdf_path` and `scanned_pdf_path` from the previous experiment.

Run:

PYTHON

```
text_pdf = fitz.open(str(text_pdf_path))
```

Check:

PYTHON

```
print("Number of pages:", len(text_pdf))
```

You should see: Number of pages: 5

Now open the scanned PDF:

PYTHON

```
scanned_pdf = fitz.open(str(scanned_pdf_path))
```

Check:

PYTHON

```
print("Number of pages:", len(scanned_pdf))
```

Expected: Number of pages: 1

5. Inspect one text-based page

Let's start small:

PYTHON

```
page = text_pdf[0]
print(page)
```

More importantly:

PYTHON

```
print("Page width :", page.rect.width)
print("Page height:", page.rect.height)
```

You'll get something similar to:

OUTPUT

```
Page width : 595.0
Page height: 842.0
```

The exact values depend on the PDF. This tells us the physical dimensions of the PDF page.

6. Extract text directly using PyMuPDF

Run:

PYTHON

```
text = page.get_text()
print(text[:1000])
```

You should see the actual text.

Now:

PYTHON

```
print("Characters:", len(text))
```

This is similar to what we saw through `document.page_content`, but now we're working directly with the PDF page.

7. Inspect text blocks

This is where things become interesting.

Run:

PYTHON

```
blocks = page.get_text("blocks")
print("Number of blocks:", len(blocks))
```

Now inspect the first block:

PYTHON

```
print(blocks[0])
```

You'll get a tuple containing information about the block conceptually similar to:

BLOCK STRUCTURE

```
(
    x0,
    y0,
    x1,
    y1,
    "some extracted text...",
    block_number,
    block_type
)
```

So a PDF page isn't simply *'one big string'*. It actually has spatial/layout information.

8. Understand the block coordinates

Let's inspect the first block more clearly:

PYTHON

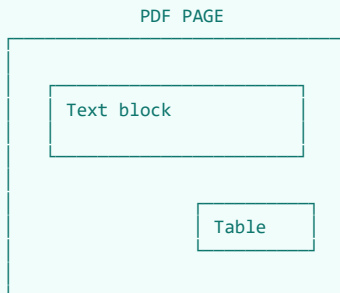
```
block = blocks[0]

print("X0:", block[0])
print("Y0:", block[1])
print("X1:", block[2])
print("Y1:", block[3])
print("Text:", block[4])
print("Block number:", block[5])
print("Block type:", block[6])
```

The coordinates describe where the block occurs on the page.

Conceptually:

SPATIAL LAYOUT



This spatial information becomes extremely useful when dealing with multi-column documents, tables, forms, invoices, reports, and complex layouts.

9. Inspect the scanned PDF page

Now inspect the scanned document:

PYTHON

```
scanned_page = scanned_pdf[0]
scanned_text = scanned_page.get_text()

print("Characters:", len(scanned_text))
print("Text:", repr(scanned_text))
```

You should see something like:

OUTPUT

```
Characters: 0
Text: ''
```

Now check blocks:

```
PYTHON
```

```
scanned_blocks = scanned_page.get_text("blocks")
print("Number of text blocks:", len(scanned_blocks))
```

You should get: Number of text blocks: 0. This is an extremely important observation.

10. Now inspect images

The scanned page visually contains our text. Where is that information? It is inside an image.

Run:

```
PYTHON
```

```
images = scanned_page.get_images(full=True)
print("Number of images:", len(images))
```

You should see: Number of images: 1

Now compare:

```
EVIDENCE COMPARISON
```

```
TEXT PDF
```

```
Text blocks → many
Images      → potentially none
```

```
SCANNED PDF
```

```
Text blocks → 0
Images      → 1
```

This is much stronger evidence than simply checking page_content.

11. Inspect the image information

Run:

```
PYTHON
```

```
image = images[0]
print(image)
```

You will see information containing xref, width, height, bits per component, and colorspace.

Let's extract some of it:

```
PYTHON
```

```
xref = image[0]
width = image[2]
height = image[3]

print("XRef   :", xref)
print("Width  :", width)
print("Height :", height)
```

Now we know: This PDF page doesn't have extractable text, but it contains a large image. That is a very strong signal that OCR may be appropriate.

12. Build a reusable page analyzer

Now let's turn what we've learned into a reusable function:

PYTHON

```
def analyze_pdf_page(page):

    text = page.get_text().strip()

    blocks = page.get_text("blocks")

    images = page.get_images(full=True)

    page_width = page.rect.width
    page_height = page.rect.height

    page_area = page_width * page_height

    character_count = len(text)
    word_count = len(text.split())

    return {
        "page_number": page.number + 1,
        "width": round(page_width, 2),
        "height": round(page_height, 2),
        "page_area": round(page_area, 2),
        "characters": character_count,
        "words": word_count,
        "text_blocks": len(blocks),
        "images": len(images),
        "has_text": character_count > 0,
        "has_images": len(images) > 0,
    }
```

13. Test it on the normal PDF

Run:

PYTHON

```
text_page_analysis = analyze_pdf_page(
    text_pdf[0]
)

text_page_analysis
```

You should get a dictionary similar to:

OUTPUT STRUCTURE

```
{
  'page_number': 1,
  'width': 595.0,
  'height': 842.0,
  'page_area': ...,
  'characters': ...,
  'words': ...,
  'text_blocks': ...,
  'images': ...,
  'has_text': True,
  'has_images': False
}
```

14. Test it on the scanned PDF

Run:

PYTHON

```
scanned_page_analysis = analyze_pdf_page(
    scanned_pdf[0]
)

scanned_page_analysis
```

You should see something conceptually like:

OUTPUT STRUCTURE

```
{
  'page_number': 1,
  'characters': 0,
  'words': 0,
  'text_blocks': 0,
  'images': 1,
  'has_text': False,
  'has_images': True
}
```

Now our detection has become much more meaningful.

15. Analyze the entire PDF

Let's analyze every page:

PYTHON

```
def analyze_pdf(pdf):

    results = []

    for page in pdf:

        results.append(
            analyze_pdf_page(page)
        )

    return results
```

Run and convert them into DataFrames:

PYTHON

```
text_pdf_analysis = analyze_pdf(text_pdf)
scanned_pdf_analysis = analyze_pdf(scanned_pdf)

import pandas as pd

text_analysis_df = pd.DataFrame(text_pdf_analysis)
text_analysis_df

scanned_analysis_df = pd.DataFrame(scanned_pdf_analysis)
scanned_analysis_df
```

16. Compare them

Conceptually you'll have:

Text-based PDF:

Page	Characters	Words	Text Blocks	Images	Has Text
1	> 0	> 0	> 0	0	True
2	> 0	> 0	> 0	0	True
3	> 0	> 0	> 0	0	True

Scanned PDF:

Page	Characters	Words	Text Blocks	Images	Has Text
1	0	0	0	1	False

This is the exact kind of information we wanted.

17. Introduce the concept of "text density"

Suppose Page area = 500,000 and Characters = 2. That's suspicious. But with Characters = 4,000, it is much more likely to be a normal text page.

We define: $\text{text_density} = \text{character_count} / \text{page_area}$

PYTHON

```
def analyze_pdf_page(page):

    text = page.get_text().strip()

    blocks = page.get_text("blocks")
    images = page.get_images(full=True)

    page_width = page.rect.width
    page_height = page.rect.height

    page_area = page_width * page_height

    character_count = len(text)
    word_count = len(text.split())

    text_density = (
        character_count / page_area
        if page_area > 0
        else 0
    )

    return {
        "page_number": page.number + 1,
        "width": round(page_width, 2),
        "height": round(page_height, 2),
        "characters": character_count,
        "words": word_count,
        "text_blocks": len(blocks),
        "images": len(images),
        "text_density": text_density,
        "has_text": character_count > 0,
        "has_images": len(images) > 0,
    }
```

Run the analysis again:

PYTHON


```
text_pdf_analysis = analyze_pdf(text_pdf)
scanned_pdf_analysis = analyze_pdf(scanned_pdf)

pd.DataFrame(scanned_pdf_analysis)
```

18. Why text density is useful

Imagine a mixed production PDF:

DOCUMENT PROFILE

```
Mixed PDF
├─ Page 1 → text (3,500 chars)
├─ Page 2 → text (4,200 chars)
├─ Page 3 → scanned / suspicious (20 chars)
├─ Page 4 → text (3,800 chars)
└─ Page 5 → image-only
```

This allows us to detect partially problematic PDFs because real documents aren't always entirely scanned.

19. Build a first "suspicious page" classifier

Now let's combine our signals:

PYTHON

```
def classify_pdf_page(page_analysis, minimum_characters=50):

    characters = page_analysis["characters"]
    images = page_analysis["images"]
    text_blocks = page_analysis["text_blocks"]

    if characters < minimum_characters:

        if images > 0:
            return "POSSIBLE_SCANNED"

        return "LOW_TEXT_CONTENT"

    if images > 0:
        return "TEXT_AND_IMAGES"

    if text_blocks > 0:
        return "TEXT_PAGE"

    return "UNKNOWN"
```

Now test the classification:

PYTHON

```
for page_analysis in scanned_pdf_analysis:

    status = classify_pdf_page(
        page_analysis
    )

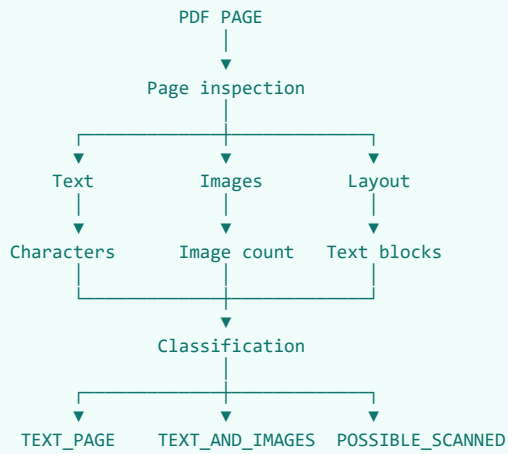
    print(
        page_analysis["page_number"],
        status
    )
```

For our scanned page, we expect: 1 POSSIBLE_SCANNED

20. Our detection architecture has evolved

We started with: 'Has text?'. Now we have:

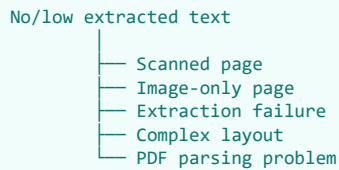
CLASSIFIER PIPELINE



21. One important limitation

POSSIBLE_SCANNED doesn't necessarily mean 'Run OCR immediately'. It means 'This page requires further investigation':

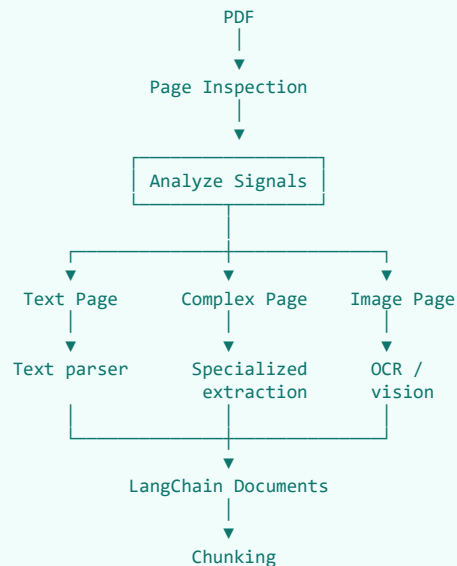
INVESTIGATION MATRIX



22. The production-oriented decision

Our eventual ingestion pipeline will look more like:

PRODUCTION INGESTION



And this is why we're spending time on detection before OCR.

23. Key Takeaways

- 1. PDF pages contain more information than just extracted text.
- 2. PyMuPDF allows us to inspect text blocks and images directly.
- 3. Text block count provides a useful extraction signal.
- 4. Image count helps identify image-heavy or scanned pages.
- 5. Character count tells us how much text was extracted.
- 6. Text density provides another signal for suspicious pages.
- 7. A PDF can contain a mixture of normal and scanned pages.
- 8. A page containing text is not automatically a correctly extracted page.
- 9. A suspicious page should be investigated before deciding to use OCR.
- 10. Deep page analysis gives us a better foundation for building an intelligent document-ingestion pipeline.
- 11. We have intentionally not installed OCR yet.

Where we are now

ROADMAP PROGRESS

Stage 1.4.2 – OCR & Complex PDF Ingestion

1.4.2.1 Text-based PDF vs Scanned PDF

1.4.2.2 Detecting Scanned Pages

1.4.2.3 Deep PDF Page Analysis

1.4.2.4 OCR Fundamentals ← Next

1.4.2.5 OCR Implementation

1.4.2.6 OCR → LangChain Documents

At this point, we stop going deeper into detection and move to **Stage 1.4.2.4 — OCR Fundamentals**. You've seen why OCR is needed, how to detect suspicious pages, and what signals can be used before invoking OCR.